

```
In [ ]: # -----
# Title: Assignment: Recommender System
# Author: Surenter Selvaraj
# Date: Feb 24, 2026
# Modified By: Surenter Selvaraj
# -----
```

Step 1: Library Initialization and Data Loading

This step imports the required libraries. It then loads the movies.csv and ratings.csv files from the local directory into Pandas DataFrames.

```
In [1]: import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Load the MovieLens dataset files
movies = pd.read_csv('movies.csv')
ratings = pd.read_csv('ratings.csv')

movies.head()
```

```
Out[1]:
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

Step 2: Data Merging and Title Standardization

The two datasets are merged into a single DataFrame using the movieId as the common key. A regular expression is applied to the title column to strip the year (eg "(1995)") and any trailing whitespace. This allows for movie lookups using just the name.

```
In [2]: df = pd.merge(ratings, movies, on='movieId')

df['title_simple'] = df['title'].str.replace(r'\s{4}', '', regex=True).str.strip()

df[['title', 'title_simple']].drop_duplicates().head()
```

```
Out[2]:
```

	title	title_simple
0	Toy Story (1995)	Toy Story
1	Grumpier Old Men (1995)	Grumpier Old Men
2	Heat (1995)	Heat
3	Seven (a.k.a. Se7en) (1995)	Seven (a.k.a. Se7en)
4	Usual Suspects, The (1995)	Usual Suspects, The

Step 3: Construction of the User-Item Matrix

A pivot table is generated to represent the relationship between movies and users. In this matrix, each row is a movie, each column is a unique user, and the cells contain the ratings. Missing values are filled with 0, indicating that a user has not rated that specific movie.

```
In [3]: movie_matrix = df.pivot_table(index='title_simple', columns='userId', values='rating').fillna(0)

movie_matrix.head()
```

Out [3]:

userId	1	2	3	4	5	6	7	8	9	10	...	601	602	603	604	605	606	607	608	609	610
title_simple																					
'71	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	4.0
'Hellboy': The Seeds of Creation	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
'Round Midnight	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
'Salem's Lot	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
'Til There Was You	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows x 610 columns

Step 4: Mathematical Similarity Computation

The system uses Cosine Similarity to calculate the distance between the rating vectors of every movie pair. A score of 1.0 indicates identical rating patterns, while 0.0 indicates no similarity. The resulting matrix is converted back into a DataFrame for querying.

```
In [5]: item_similarity = cosine_similarity(movie_matrix)

sim_df = pd.DataFrame(item_similarity, index=movie_matrix.index, columns=movie_matrix.index)

sim_df.head()
```

Out [5]:

title_simple	'71	'Hellboy': The Seeds of Creation	'Round Midnight	'Salem's Lot	'Til There Was You	'Tis the Season for Love	'burbs, The	'night Mother	(500) Days of Summer	*batteries not included	...	Zulu	[
title_simple													
'71	1.0	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	0.0	0.141653	0.0	...	0.0	0.34
'Hellboy': The Seeds of Creation	0.0	1.000000	0.707107	0.000000	0.000000	0.0	0.000000	0.0	0.000000	0.0	...	0.0	0.00
'Round Midnight	0.0	0.707107	1.000000	0.000000	0.000000	0.0	0.176777	0.0	0.000000	0.0	...	0.0	0.00
'Salem's Lot	0.0	0.000000	0.000000	1.000000	0.857493	0.0	0.000000	0.0	0.000000	0.0	...	0.0	0.00
'Til There Was You	0.0	0.000000	0.000000	0.857493	1.000000	0.0	0.000000	0.0	0.000000	0.0	...	0.0	0.00

5 rows × 9446 columns

Step 5: Recommendation Logic Implementation

A function is defined to retrieve the top 10 most similar movies for a given input. It identifies the column corresponding to the input movie, sorts all other movies by their similarity score in descending order, and excludes the input movie from the final list.

```
In [6]: def get_recommendations(movie_name):
        if movie_name not in sim_df.index:
            return f"The movie '{movie_name}' was not found in the dataset."

        recommendations = sim_df[movie_name].sort_values(ascending=False).iloc[1:11]

        return recommendations.to_frame(name='Similarity Score')
```

Step 6: Execution and Results

The following examples demonstrate the system's ability to recommend titles across different genres using only the movie name.

```
In [ ]: print("Recommendations for Toy Story:")
        print(get_recommendations('Toy Story'))

        print("\n" + "="*30 + "\n")

        print("Recommendations for Matrix, The:")
        print(get_recommendations('Matrix, The'))

        print("\n" + "="*30 + "\n")

        print("Recommendations for Pulp Fiction:")
        print(get_recommendations('Pulp Fiction'))
```

Recommendations for Toy Story:

	Similarity Score
title_simple	
Toy Story 2	0.572601
Jurassic Park	0.565637
Independence Day (a.k.a. ID4)	0.564262
Star Wars: Episode IV – A New Hope	0.557388
Forrest Gump	0.547096
Lion King, The	0.541145
Star Wars: Episode VI – Return of the Jedi	0.541089
Mission: Impossible	0.538913
Groundhog Day	0.534169
Back to the Future	0.530381

=====

Recommendations for Matrix, The:

	Similarity Score
title_simple	
Fight Club	0.713937
Star Wars: Episode V – The Empire Strikes Back	0.700935
Saving Private Ryan	0.679615
Star Wars: Episode IV – A New Hope	0.663447
Star Wars: Episode VI – Return of the Jedi	0.660984
Lord of the Rings: The Fellowship of the Ring, The	0.654128
Sixth Sense, The	0.651307
Lord of the Rings: The Return of the King, The	0.640203
Raiders of the Lost Ark (Indiana Jones and the ...	0.631107
Gladiator	0.626407

=====

Recommendations for Pulp Fiction:

	Similarity Score
title_simple	
Silence of the Lambs, The	0.709382
Shawshank Redemption, The	0.702366
Seven (a.k.a. Se7en)	0.697654
Forrest Gump	0.685544
Usual Suspects, The	0.672616
Braveheart	0.627621
Fight Club	0.623220
Fargo	0.610349

Terminator 2: Judgment Day	0.610284
Reservoir Dogs	0.603955

System Summary

The recommender system utilizes Item-Based Collaborative Filtering. This approach builds a model based on user-item interactions rather than external metadata. By analyzing the patterns in how different users rate items, the system can identify "latent" similarities between movies that might not share the same genre or cast.

References

- Item-Based Collaborative Filtering in Python (Towards Data Science) : <https://towardsdatascience.com/item-based-collaborative-filtering-in-python-91f747200fab/>
- Build Your Own Recommender System (Analytics Vidhya) : <https://www.analyticsvidhya.com/blog/2021/05/item-based-collaborative-filtering-build-your-own-recommender-system/>